

Compétences Techniques

Chris Reny @thvlddd

November 2025

1 Compétences Techniques de l'Investigateur Numérique : Fondements Scientifiques et Référentiel Pratique

1.1 Introduction : L'Excellence Technique comme Fondation

1.1.1 Le Défi de la Polyvalence Technique

L'investigation numérique moderne exige une polyvalence technique sans précédent. Contrairement aux disciplines forensiques traditionnelles (balistique, empreintes digitales, ADN) où l'expert peut se spécialiser étroitement, l'investigateur numérique doit maîtriser un spectre technologique vaste et en évolution rapide :

Réalité du terrain :

- Une investigation typique mobilise 8-12 domaines techniques différents (systèmes d'exploitation, réseaux, bases de données, cryptographie, mobile, cloud...)
- 78% des investigations échouent ou produisent conclusions erronées par lacunes techniques (SANS Digital Forensics Survey, 2024)
- Le délai moyen entre émergence d'une technologie et son exploitation criminelle : 18 mois
- L'obsolescence des compétences techniques : 30-40% tous les 3 ans (vs 10-15% pour compétences non techniques)

Le paradoxe du généraliste-spécialiste :

L'investigateur doit être simultanément :

- **Généraliste** : Comprendre architecture globale (systèmes, réseaux, applications) pour naviguer investigations complexes
- **Spécialiste** : Maîtriser en profondeur 2-3 domaines pour produire analyses forensiques robustes
- **Apprenant perpétuel** : Actualiser continuellement compétences face à évolution technologique

1.1.2 Statistiques Critiques

Impact des compétences techniques sur la qualité investigative :

- Investigateurs maîtrisant ≥10 compétences techniques clés : taux de résolution 73% vs 41% pour ≤6 compétences (HTCIA, 2023)
- 61% des preuves numériques contestées en justice le sont pour erreurs techniques (mauvaise acquisition, analyse incorrecte, chaîne de traçabilité rompue) — source : ENFSI Digital Forensics, 2024
- Coût moyen d’une erreur technique majeure en investigation d’entreprise : 450k€ (perte preuves, réinvestigation, contentieux)
- 89% des investigateurs seniors citent profondeur technique comme différence principale avec juniors

1.1.3 Objectifs d’Apprentissage du Chapitre

À l’issue de ce chapitre, l’étudiant sera capable de :

1. **Comprendre** les fondements scientifiques (informatique théorique, cryptographie, sciences forensiques) sous-tendant les techniques d’investigation
2. **Identifier** les 18 compétences techniques essentielles organisées en 5 blocs thématiques
3. **Évaluer** son niveau actuel de maîtrise technique via grilles d’auto-diagnostic
4. **Appliquer** méthodologies forensiques rigoureuses garantissant recevabilité juridique
5. **Maîtriser** outils forensiques standards (open source et commerciaux)
6. **Développer** scripts d’automatisation pour efficacité et reproductibilité
7. **Maintenir** compétences techniques à jour face à évolution rapide
8. **Construire** trajectoire de spécialisation progressive (généraliste → expert de domaine)

1.1.4 Structure Pédagogique du Chapitre

Le chapitre s’organise en trois parties complémentaires :

Partie I – Fondements Scientifiques (Section 2) : Présentation des bases théoriques en informatique, cryptographie, mathématiques appliquées et sciences forensiques qui fondent la pratique investigative technique. Cette section établit la légitimité scientifique de la discipline.

Partie II – Référentiel des Compétences Techniques (Sections 3-7) : Développement systématique des 18 compétences techniques essentielles, organisées en 5 blocs :

1. **Bloc Fondamental** : Architecture systèmes et réseaux (socle de compréhension)
2. **Bloc Forensique** : Acquisition, préservation et analyse des preuves
3. **Bloc Analytique** : Reverse engineering et analyse de malware
4. **Bloc Sécurité** : Cryptographie et sécurité offensive/défensive
5. **Bloc Automatisation** : Programmation, scripting et développement d'outils

Partie III – Plan de Développement Technique (Section 8) : Trajectoires d'apprentissage, certifications professionnelles, laboratoires pratiques et plan de spécialisation sur 36 mois.

Chaque compétence technique sera présentée selon le schéma unifié :

- **Définition technique précise** avec vocabulaire normalisé
- **Justification pour l'investigation** (cas d'usage concrets)
- **Ancrage théorique** (renvois aux fondements Section 2)
- **Niveaux de maîtrise** (Débutant / Intermédiaire / Avancé / Expert)
- **Outils de référence** (open source et commerciaux)
- **Exercices pratiques** progressifs avec solutions
- **Erreurs techniques fréquentes** et comment les éviter
- **Checklist de validation** technique
- **Veille technologique** (ressources d'actualisation)

Prérequis pour ce chapitre :

- Connaissance de base en informatique (utilisation courante systèmes, réseaux)
- Compréhension concepts fondamentaux : fichiers, processus, paquets réseau, bases de données
- Aucune expertise préalable requise (chapitre construit progressivement)

2 PARTIE I – Fondements Scientifiques et Théoriques

2.1 Introduction : La Technique au Service de la Science

L'investigation numérique n'est pas simple bidouillage d'outils forensiques. C'est une discipline scientifique appliquée s'appuyant sur :

- **Informatique théorique** : Architecture des systèmes, théorie de la complexité

- **Mathématiques appliquées** : Cryptographie, théorie de l'information, probabilités
- **Sciences forensiques** : Méthodologie scientifique, chaîne de traçabilité, reproductibilité
- **Droit de la preuve** : Standards de recevabilité (Daubert, Frye), expertise scientifique

Cette section établit les fondations théoriques permettant de :

1. Comprendre *pourquoi* les techniques fonctionnent (pas seulement *comment*)
2. Adapter techniques à situations nouvelles (transfert de connaissances)
3. Développer nouvelles méthodes face à technologies émergentes
4. Défendre scientifiquement conclusions en contexte contradictoire (expertise judiciaire)
5. Distinguer pratiques scientifiquement valides des pseudo-sciences

2.2 Fondements en Informatique Théorique

2.2.1 Architecture des Systèmes de Calcul

Le modèle de von Neumann (1945) L'architecture fondamentale des ordinateurs modernes repose sur le modèle de von Neumann, établissant cinq composants essentiels :

1. **Unité Arithmétique et Logique (ALU)** : Effectue opérations
2. **Unité de Contrôle** : Décode et orchestre instructions
3. **Mémoire** : Stocke données ET programmes (concept clé)
4. **Entrées** : Réception données externes
5. **Sorties** : Transmission résultats

Principe révolutionnaire : Programme stocké en mémoire (comme données), modifiable dynamiquement. Avant : circuits câblés fixes.

Implications forensiques :

- **Volatilité mémoire** : Données perdues à extinction (RAM). Artefacts temporaires critiques.
- **Séquentialité** : Instructions exécutées séquentiellement → reconstruction chronologie via analyse CPU registers, instruction pointer
- **Unité programme-données** : Code malveillant stocké comme données banales → difficulté détection statique

Application CTF-1 (Architecture systèmes) : Comprendre architecture von Neumann permet :

- Identifier artefacts forensiques pertinents (registres CPU, stack, heap)
- Reconstruire séquence d'exécution
- Distinguer code légitime vs malveillant en mémoire

Hierarchie mémoire et principes de localité Hierarchie mémoire (du plus rapide au plus lent) :

1. **Registres CPU** (cycles : 1, taille : octets)
2. **Cache L1/L2/L3** (cycles : 1-10, taille : Ko-Mo)
3. **RAM** (cycles : 100, taille : Go)
4. **SSD/HDD** (cycles : 100k-10M, taille : To)
5. **Réseau/Cloud** (cycles : >10M, taille :)

Principes de localité (Denning, 1968) :

Localité temporelle Donnée récemment accédée sera probablement ré-accédée bientôt

Localité spatiale Données proches en mémoire seront probablement accédées ensemble

Implications forensiques CTF-2 (Analyse mémoire) :

- **Artefacts cache** : Données récentes (temporelle) dans cache CPU/disque. Analyse cache = fenêtre sur activité récente.
- **Reconstruction activité** : Localité spatiale → fichiers liés souvent contigus. Fragmentation = anomalie potentielle.
- **Volatilité différentielle** : Registres ; RAM ; SSD en persistance. Priorisation acquisition : plus volatil d'abord.

Systèmes d'exploitation : Abstractions et gestion des ressources Rôle du système d'exploitation :

1. **Abstraction matérielle** : Fournit interface uniforme (API) cachant complexité hardware
2. **Gestion ressources** : CPU (scheduling), mémoire (pagination), I/O (drivers)
3. **Isolation et sécurité** : Séparation processus (mémoire virtuelle), contrôle accès (permissions)

4. **Persistence** : Système de fichiers structurant stockage

Concepts clés forensiques :

Processus vs Thread Processus = instance programme (espace mémoire propre). Thread = unité d'exécution dans processus (partage mémoire). → Analyse forensique : dump processus capture threads + mémoire partagée.

Mémoire virtuelle Chaque processus voit espace mémoire isolé (protection). OS mappe mémoire virtuelle → physique via tables de pages. → Forensique : reconstruction mapping nécessaire pour analyser dump mémoire brute.

Système de fichiers Structure logique (FAT, NTFS, ext4, APFS) organisant données sur stockage physique. Métadonnées : timestamps (MAC times : Modified, Accessed, Created), permissions, ownership. → Forensique : timeline reconstruction, détection anti-forensique (timestamp manipulation).

Application CTf-1 et CTf-3 :

- Comprendre abstractions OS permet identifier couches où artefacts résident
- Maîtriser mémoire virtuelle essentiel pour analyse dumps mémoire (CTf-3)
- Connaître structures filesystems critique pour forensique disque (CTf-2)

2.2.2 **Théorie de l'Information et Entropie**

Théorie de Shannon (1948) Claude Shannon, dans A Mathematical Theory of Communication , établit fondements mathématiques de l'information.

Entropie de Shannon : Mesure de l'incertitude/information dans source de données.

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

Où $p(x_i)$ = probabilité du symbole x_i .

Interprétation :

- $H = 0$: Totalement prévisible (pas d'information). Exemple : texte avec seul caractère A .
- $H = \log_2(n)$: Maximum entropie (distribution uniforme). Exemple : données aléatoires, chiffrement fort.

Applications forensiques CTf-6 (Cryptographie) et CTf-8 (Analyse malware) :

1. **Détection de chiffrement :**

- Données chiffrées (AES, etc.) : entropie proche maximale (8 bits/byte pour byte data)
- Texte clair : entropie faible (anglais 4.5 bits/char, français 3.9)
- **Outil** : Calculer entropie fichiers suspects. $H > 7.5$ bits/byte $\rightarrow$ probablement chiffré/compressé.

2. Détection de malware packés :

- Malware souvent compressé/chiffré (UPX, custom packers) pour échapper détection
- Entropie sections exécutables : code normal 5-6, packé 7.5-8
- **Signature entropique** : Section .text avec entropie anormalement haute = suspect

3. Stéganographie :

- Données cachées dans image doivent maintenir entropie naturelle de l'image
- Stéganalyse : détecter anomalies entropiques (LSB steganography augmente légèrement entropie)

Exemple pratique :

```
# Calcul entropie avec Python
import math
from collections import Counter

def entropy(data):
    if not data:
        return 0
    entropy = 0
    counter = Counter(data)
    for count in counter.values():
        p = count / len(data)
        entropy -= p * math.log2(p)
    return entropy

# Analyse fichier suspect
with open('suspect.exe', 'rb') as f:
    data = f.read()
    h = entropy(data)
    print(f"Entropie : {h:.2f} bits/byte")
    if h > 7.5:
        print("ALERTE : Probablement chiffré/compressé")
```

Compression et redondance **Théorème de Shannon sur la compression** : On ne peut compresser des données en dessous de leur entropie sans perte d'information.

Implication forensique CTf-2 :

- Fichiers compressés (ZIP, RAR, 7z) ont entropie élevée (redondance éliminée)
- Tentative compression données déjà chiffrées : ratio 1 (pas de gain)
- **Test forensique** : Comprimer fichier suspect. Si taille inchangée → probablement déjà chiffré.

2.2.3 Théorie de la Complexité et Calculabilité

Classes de complexité P, NP, NP-complet Définitions :

P Problèmes résolubles en temps polynomial (faciles). Ex : tri, recherche binaire.

NP Problèmes dont solution est vérifiable en temps polynomial. Ex : factorisation, SAT.

NP-complet Problèmes NP les plus durs . Si un NP-complet est dans P, alors $P=NP$.

Conjecture PNP : Largement crue, non prouvée. Implique : certains problèmes intrinsèquement difficiles.

Application CTf-6 (Cryptographie) :

Sécurité de nombreux systèmes cryptographiques repose sur problèmes présumés difficiles :

- **RSA** : Factorisation grands nombres (NP, probablement hors P)
- **Diffie-Hellman, ECC** : Logarithme discret (difficile)
- **Cassage** : Si $P=NP$ prouvé, cryptographie actuelle s'effondre

Implications pratiques :

- Cassage bruteforce RSA-2048 : 2^{2048} opérations impossible (univers mourra avant)
- Mais vérification signature RSA : rapide (polynomial)
- → Asymétrie attaque/défense fondamentale en crypto

Application CTf-8 (Reverse engineering) :

- Obfuscation code : transforme programme clair en équivalent obscur
- Certaines obfuscations basées sur problèmes NP-complets (équivalence programmes)
- → Désobfuscation peut être intrinsèquement difficile (pas juste question d'outils)

2.3 Fondements en Cryptographie Mathématique

2.3.1 Théorie des Nombres et Cryptographie

Arithmétique modulaire et corps finis Congruence modulo n : $a \equiv b \pmod{n}$ si $n \mid (a - b)$.

Propriétés fondamentales :

- $(a + b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$
- $(a \times b) \bmod n = ((a \bmod n) \times (b \bmod n)) \bmod n$
- Inverse multiplicatif : $a \times a^{-1} \equiv 1 \pmod{n}$ si $\gcd(a, n) = 1$

Application RSA (CTf-6) :

RSA repose sur :

1. Choisir p, q premiers grands. $n = p \times q$.
2. $\phi(n) = (p - 1)(q - 1)$ (fonction d'Euler)
3. Clé publique : (e, n) où $\gcd(e, \phi(n)) = 1$
4. Clé privée : $d = e^{-1} \bmod \phi(n)$
5. Chiffrement : $c = m^e \bmod n$
6. Déchiffrement : $m = c^d \bmod n$

Sécurité : Cassage nécessite factoriser $n = p \times q$ (difficile si p, q grands).

Forensique :

- Clés RSA faibles (petits p, q) factorisables $\rightarrow$ récupération clé privée
- Analyse entropy peut révéler clés faibles (entropy insuffisante)

Logarithme discret et courbes elliptiques **Problème du logarithme discret :** Étant donnés $g, g^x \bmod p$, trouver x (difficile).

Courbes elliptiques (ECC) :

Courbe : $y^2 = x^3 + ax + b \pmod{p}$

Addition de points définie géométriquement. Problème : $Q = kP$, connaissant P, Q , trouver k (difficile).

Avantage ECC vs RSA : Sécurité équivalente avec clés plus courtes.

- RSA-2048 ECC-224 (sécurité équivalente)
- ECC : calculs plus rapides, clés plus petites $\rightarrow$ adoption croissante (Bitcoin, Signal, TLS 1.3)

Application CTf-6 :

- Analyser protocoles cryptographiques modernes nécessite comprendre ECC
- Implémentations ECC vulnérables (timing attacks, courbes faibles)
- Forensique blockchain (Bitcoin) : signatures ECDSA

2.3.2 Sécurité Computationnelle et Preuves

Modèles de sécurité : IND-CPA, IND-CCA Indistinguishability under Chosen Plaintext Attack (IND-CPA) :

Adversaire ne peut distinguer chiffré de m_0 vs m_1 même en choisissant m_0, m_1 et observant multiples chiffréments.

Indistinguishability under Chosen Ciphertext Attack (IND-CCA) :

Adversaire peut aussi déchiffrer chiffrés de son choix (sauf le chiffré challenge).

Hiérarchie : IND-CCA $\supset$ IND-CPA $\supset$ sécurité de base

Application CTf-6 :

- Schémas cryptographiques doivent atteindre au minimum IND-CPA
- ECB mode (Electronic Codebook) : NE satisfait PAS IND-CPA (patterns visibles)
- CBC, CTR, GCM : satisfont IND-CPA (ou mieux)
- **Analyse forensique** : Identifier mode chiffrement utilisé $\rightarrow$ évaluer robustesse

Fonctions de hachage cryptographiques Propriétés requises :

1. **Résistance aux préimages** : Étant donné h , difficile trouver m tel que $H(m) = h$
2. **Résistance aux secondes préimages** : Étant donné m_1 , difficile trouver $m_2 \neq m_1$ tel que $H(m_1) = H(m_2)$
3. **Résistance aux collisions** : Difficile trouver $m_1 \neq m_2$ tel que $H(m_1) = H(m_2)$

Attaque des anniversaires : Trouver collision nécessite $O(2^{n/2})$ essais pour hash n bits (vs $O(2^n)$ pour préimage).

Exemple : MD5 (128 bits) : collision trouvable en 2^{64} faisable (2004).
SHA-256 : 2^{128} infaisable.

Applications forensiques CTf-2, CTf-6 :

- **Intégrité preuves** : Hash (SHA-256) de disque acquis. Rehash après analyse. Si identique $\rightarrow$ intégrité préservée.
- **Identification fichiers** : Hash unique = empreinte. Bases de données (NSRL, VirusTotal) identifient fichiers connus.
- **Détection altération** : Changement 1 bit $\rightarrow$ hash totalement différent (effet avalanche)
- **Déduplication** : Mêmes hashes $\rightarrow$ fichiers identiques (gain espace stockage preuves)

2.4 Fondements en Sciences Forensiques

2.4.1 Le Principe d'Échange de Locard

Énoncé (Locard, 1920) : Tout contact laisse une trace.

Originellement pour forensique physique (fibres, traces), mais universellement applicable.

Transposition numérique :

Toute action numérique laisse traces (artefacts) :

- **Exécution programme** → Logs système, prefetch (Windows), registres, mémoire résiduelle
- **Navigation web** → Cache navigateur, cookies, historique, DNS cache
- **Accès fichier** → Timestamps (MAC), journaux filesystem, signatures mémoire
- **Communication réseau** → Logs firewall, captures paquets (pcap), logs proxy
- **Modification système** → Event logs, registres de configuration, auditing

Implications pour TOUTES compétences techniques :

1. **Aucune action sans trace** : Même attaquant sophistiqué laisse artefacts (challenge : les trouver)
2. **Multiples sources** : Une action génère traces dans plusieurs systèmes → corroboration possible
3. **Fenêtre temporelle** : Traces volatiles (mémoire, cache) vs persistantes (disque, logs centralisés)
4. **Anti-forensique limité** : Effacer TOUTES traces extrêmement difficile (Locard : impossible perfection)

Exemple appliqué :

Exécution malware sur Windows :

- **Disque** : Fichier exécutable, prefetch (.pf), Amcache, ShimCache
- **Mémoire** : Processus actif, DLLs injectées, handles réseau
- **Registre** : Clés persistance (Run, RunOnce, Services)
- **Réseau** : Connexions établies, DNS queries, trafic C2
- **Logs** : Sysmon, Windows Event Logs (4688, 7045, etc.)

Supprimer fichier exécutable N'efface PAS les autres traces → reconstruction possible.

2.4.2 Chaîne de Traçabilité (Chain of Custody)

Définition : Documentation chronologique complète de collecte, transfert, analyse, stockage d'une preuve.

Objectif : Garantir que preuve présentée en justice est identique à preuve collectée, non altérée, non contaminée.

Composants d'une chaîne valide :

1. **Identification unique :** Numéro de scellé, description précise
2. **Documentation initiale :**
 - Date/heure collecte (timestamp UTC)
 - Lieu précis
 - Personne ayant collecté (nom, fonction)
 - État initial (photos, description)
 - Hash cryptographique (MD5+SHA256)
3. **Transferts :** Chaque transfert documenté
 - De : Personne A (signature)
 - À : Personne B (signature)
 - Date/heure
 - Motif transfert
 - Vérification intégrité (rehash)
4. **Stockage sécurisé :**
 - Lieu physiquement sécurisé (coffre, salle contrôlée)
 - Accès tracé (badge, registre)
 - Conditions environnementales (température, humidité si pertinent)
5. **Analyses :** Chaque analyse documentée
 - Analyste (nom, qualification)
 - Date/heure début/fin
 - Outils utilisés (nom, version)
 - Hash pré/post analyse (vérifier non-altération)

Rupture de chaîne → inadmissibilité potentielle :

- Gap temporel non expliqué
- Transfert non documenté
- Hash différent (altération)

- Accès non autorisé

Application CTf-2 (Acquisition forensique) :
Formulaire type de chaîne de traçabilité :

CHAIN OF CUSTODY FORM

Evidence ID : 2025-INV-001-HDD01
 Description : Seagate 2TB HDD, S/N : ZA123456
 Case : Investigation intrusion réseau EntrepriseX

COLLECTION

Date/Time : 2025-11-07 14:32 UTC
 Location : Serveur Salle A, Rack 3, Slot 12
 Collected by : Jean Dupont, Senior Investigator, Badge #4521
 Initial State : Powered OFF, connected SATA, intact seals
 Hash MD5 : d41d8cd98f00b204e9800998ecf8427e
 Hash SHA256 : e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
 Photos : IMG_001.jpg to IMG_005.jpg
 Sealed : Evidence bag #2025-001, tamper-evident seal

TRANSFERS

[1] From : Jean Dupont	To : Marie Martin (Lab Tech)
Date : 2025-11-07 16:45	Reason : Transport to lab
Signatures : _____	_____
[2] From : Marie Martin	To : Paul Bernard (Analyst)
Date : 2025-11-08 09:15	Reason : Forensic analysis
Hash verified : MATCH	Signatures : _____

ANALYSIS LOG

Analyst : Paul Bernard, GCFA Certified
 Start : 2025-11-08 09:30 End : 2025-11-08 17:20
 Tools : FTK Imager 4.7.1, Autopsy 4.20.0
 Method : Write-blocked acquisition, logical+physical imaging
 Hash post-acquisition : MATCH original
 Notes : See analysis report INV-001-REPORT-001

STORAGE

Location : Evidence Locker B-12, Room 204
 Access log : [Timestamps + Badge scans]

2.4.3 Méthodologie Scientifique Appliquée

Le cycle hypothético-déductif en investigation Étapes (inspirées de Popper) :

1. **Observation** : Incident détecté (ex : accès non autorisé, exfiltration données)
2. **Hypothèse** : Formulation explication possible (Employé X a compromis système via credential theft)
3. **Prédictions** : Si H vraie, on devrait observer :
 - Logs authentification montrant accès X aux heures suspectes
 - Trafic réseau anormal depuis poste X
 - Présence outils malveillants sur poste X
 - Données cibles accessibles par X
4. **Test** : Analyse forensique pour vérifier prédictions
5. **Évaluation** :
 - Si prédictions confirmées : H corroborée (confiance augmente)
 - Si prédictions infirmées : H réfutée → formuler nouvelle hypothèse
6. **Réplication** : Analyse par second investigateur (vérification)

Application à CTF-15 (Méthodologie investigative) :

Investigation scientifique partir à la pêche aux artefacts. Processus structuré :

- Hypothèses multiples dès le début (éviter biais confirmation)
- Recherche active de contre-preuves (falsification poppérienne)
- Documentation rigoureuse permettant réplication
- Conclusions proportionnées à preuves (pas sur-affirmation)

Standards Daubert et Frye pour expertise scientifique **Standard Frye (1923, USA)** : Méthode doit être généralement acceptée dans communauté scientifique pertinente.

Standard Daubert (1993, USA) : Juge évalue fiabilité scientifique selon 5 critères :

1. **Testabilité** : Méthode peut-elle être testée/réfutée ?
2. **Peer review** : Méthode publiée/revue par pairs ?
3. **Taux d'erreur** : Taux d'erreur connu et acceptable ?
4. **Standards** : Existence de standards contrôlant technique ?
5. **Acceptation** : Degré d'acceptation dans communauté pertinente ?

Application à forensique numérique :

Méthodes Daubert-compliant :

- Analyse hash files (SHA-256) : testable, peer-reviewed, taux erreur connu (0), standards (NIST), accepté universellement
- Analyse timeline via timestamps : méthodologie documentée, reproductible, acceptée

Méthodes problématiques :

- Analyse comportementale utilisateur sans méthodologie claire
- Outils propriétaires sans documentation algorithmes
- Conclusions basées sur expérience sans méthodologie reproductible

Implication pour CTf-14, CTf-15 :

L'investigateur doit :

- Utiliser méthodes scientifiquement validées
- Documenter méthodologie de façon reproductible
- Connaître taux d'erreur outils utilisés
- Pouvoir défendre scientifiquement conclusions en contre-interrogatoire

2.5 Fondements en Théorie des Réseaux

2.5.1 Modèle OSI et TCP/IP

Modèle OSI (7 couches) :

1. **Physique** : Bits sur câble (voltage, fréquences)
2. **Liaison** : Trames entre nœuds adjacents (Ethernet, MAC)
3. **Réseau** : Routage paquets (IP, ICMP)
4. **Transport** : Communication bout-en-bout (TCP, UDP)
5. **Session** : Gestion sessions (NetBIOS, RPC)
6. **Présentation** : Encodage données (ASCII, JPEG, chiffrement)
7. **Application** : Protocoles applicatifs (http, DNS, SMTP)

Modèle TCP/IP (4 couches, pratique) :

1. **Accès réseau** : Physique + Liaison
2. **Internet** : IP

3. **Transport** : TCP/UDP

4. **Application** : http, DNS, etc.

Encapsulation : Chaque couche ajoute en-tête (header).

Exemple http :

Application : http GET /index.html
↓ + TCP header (port 80, seq, ack)
Transport : TCP segment
↓ + IP header (src IP, dst IP)
Internet : IP packet
↓ + Ethernet header (src MAC, dst MAC)
Liaison : Ethernet frame
↓
Physique : Bits sur câble

Application CTf-4 (Forensique réseau) :

- **Analyse multi-couches** : Capture pcap contient toutes couches → re-construction complète communication
- **Protocoles applicatifs** : Comprendre http, DNS, SMB, etc. pour identifier activités malveillantes
- **Anomalies de couche** : Trafic malveillant souvent viole standards protocolaires → détection par analyse conformité

2.5.2 Protocoles Fondamentaux

TCP : Transmission Control Protocol Caractéristiques :

- **Orienté connexion** : 3-way handshake (SYN, SYN-ACK, ACK)
- **Fiable** : Acquittements, retransmission si perte
- **Ordonné** : Numéros de séquence garantissent ordre
- **Contrôle de flux** : Window size évite saturation

Forensique TCP (CTf-4) :

- **Reconstruction sessions** : Wireshark Follow TCP Stream reconstitue conversation
- **Détection port scanning** : Multiples SYN sans ACK = scan (nmap)
- **Covert channels** : Données cachées dans champs TCP (ISN, timestamps, options)

DNS : Domain Name System **Fonction :** Résolution noms (google.com)
→ adresses IP (142.250.x.x).

Hiérarchie : Root servers → TLD (.com, .fr) → Authoritative servers

Types d'enregistrements :

- **A** : IPv4 address
- **AAAA** : IPv6 address
- **CNAME** : Alias
- **MX** : Mail exchange
- **TXT** : Texte arbitraire (SPF, DKIM, etc.)

Forensique DNS (CTf-4) :

- **C2 malware** : Résolution domaines suspects révèle infrastructure attaquant
- **DNS tunneling** : Données exfiltrées via queries DNS (TXT records)
- **DGA (Domain Generation Algorithm)** : Malware génère domaines aléatoires journalièrement (Conficker, Zeus)
- **Timeline** : DNS cache local (Windows : 'ipconfig /displaydns') montre résolutions récentes

Exemple DGA detection :

Domaines légitimes : 'google.com', 'facebook.com' (entropie lexicale faible, mots reconnaissables)

Domaines DGA : 'qwerthjklasz.com', 'poiuytrewqlkjh.net' (entropie élevée, non-sens)

Détection : Calculer entropie nom domaine (Shannon). Si $H > 4.5$ bits/char → suspect DGA.

2.6 Synthèse Intégrative : Fondements → Compétences

Le tableau suivant cartographie les liens entre fondements théoriques et compétences techniques pratiques.

2.7 Message aux Étudiants : La Théorie comme Levier

Cette section théorique peut sembler abstraite. Pourquoi étudier arithmétique modulaire, entropie de Shannon, modèle OSI ?

Trois raisons fondamentales :

1. **Compréhension profonde** : Théorie explique *pourquoi* techniques fonctionnent. Sans théorie, vous êtes utilisateur d'outils. Avec théorie, vous êtes expert capable d'innover.

2. **Adaptabilité** : Technologies évoluent rapidement (tous les 2-3 ans). Théorie est stable (arithmétique modulaire identique depuis 2000 ans). Maîtriser théorie permet adapter pratiques à nouvelles technologies.

Exemple : Nouveau filesystem propriétaire apparaît. Sans théorie : attendre outil forensique commercial. Avec théorie : comprendre principes généraux filesystems → développer propres outils/méthodes.

3. **Crédibilité professionnelle** : En expertise judiciaire, avocat adverse challenge méthodologie. Sans fondement théorique, expert discrédité (Vous avez cliqué sur boutons sans comprendre ?). Avec théorie, expert défend scientifiquement choix (Daubert-compliant).

Stratégie d'apprentissage :

- Ne pas mémoriser formules par cœur (disponibles en référence)
- Comprendre *concepts* et *relations* entre concepts
- Revisiter cette section après pratique (concepts abstraits deviennent concrets)
- Utiliser théorie pour *critiquer* outils/méthodes existants (Cet outil est-il scientifiquement valide ?)

Les sections suivantes (Partie II) traduiront ces fondements en compétences pratiques opérationnelles.

3 PARTIE II – Référentiel des 18 Compétences Techniques Essentielles

3.1 Vue d'Ensemble et Organisation

Le tableau suivant présente la cartographie complète des 18 compétences techniques structurant l'excellence investigative en forensique numérique.

3.2 Priorisation et Trajectoires d'Apprentissage

Parcours recommandé sur 36 mois :

Phase 1 (Mois 1-12) – Socle Fondamental :

- **Priorité absolue** : CTf-1, CTf-2, CTf-5, CTf-15
- **Objectif** : Maîtriser bases systèmes, acquisition conforme, automation basique
- **Validation** : Certification GCFE (GIAC Certified Forensic Examiner) ou équivalent

Phase 2 (Mois 13-24) – Approfondissement Forensique :

- **Priorité** : CTf-3, CTf-4, CTf-6, CTf-7, CTf-16, CTf-18
- **Objectif** : Forensique avancée (mémoire, réseau, mobile), corrélation
- **Validation** : GCFA (GIAC Certified Forensic Analyst)

Phase 3 (Mois 25-36) – Spécialisation :

Choisir 1-2 parcours selon intérêt/opportunités :

Parcours A – Analyse Malware :

- **Focus** : CTf-9, CTf-10, CTf-12, CTf-17
- **Validation** : GREM (GIAC Reverse Engineering Malware)

Parcours B – Sécurité Offensive :

- **Focus** : CTf-11, CTf-13, CTf-14
- **Validation** : GPEN (GIAC Penetration Tester) ou OSCP

Parcours C – Forensique Avancée :

- **Focus** : CTf-8 (cloud), CTf-17 (outils custom), approfondissement CTf-3, CTf-7
- **Validation** : GCFA + spécialisations (iOS, Android)

4 Bloc 1 – Compétences Fondamentales

4.1 CTf-1 : Architecture et Internals des Systèmes d'Exploitation

4.1.1 Définition Technique

Compréhension approfondie de l'architecture interne des systèmes d'exploitation majeurs (Windows 10/11, Linux, macOS), incluant : gestion processus et threads, mémoire virtuelle, système de fichiers, registre (Windows), scheduling, gestion I/O, modèle de sécurité et permissions.

4.1.2 Pourquoi c'est Critique en Investigation

Cas réel illustratif :

Investigation d'une intrusion sur serveur Windows. Logs événements montrent accès administrateur légitime. Investigateur junior conclut : pas d'intrusion. Investigateur senior connaissant internals Windows :

- Examine registre : clé 'HKLMNTFile Execution Options'
- Découvre : debugger attaché à 'sethc.exe' (sticky keys)
- Technique : Attaquant remplace sticky keys par cmd.exe → accès admin sans authentification (appui 5× Shift)

- Conclusion : intrusion confirmée, technique sophistiquée

Sans connaissance internals, l'intrusion passe inaperçue.

Impact mesurable :

- 73% des artefacts forensiques critiques résident dans structures internes OS (registre, prefetch, ShimCache, Amcache) invisibles sans compréhension internals
- Investigateurs maîtrisant internals identifient $2.8\times$ plus d'artefacts de persistance que ceux utilisant uniquement outils automatiques (SANS Survey, 2023)

4.1.3 Ancrage Théorique

- **Von Neumann** : Séparation (relative) code/données → OS stocke métadonnées multiples sur exécutions
- **Mémoire virtuelle** : Abstraction isolation → forensique nécessite comprendre mapping virtuel→physique
- **Principe Locard** : OS multiplie traces (logs, caches, registres) de toute action

4.1.4 Niveaux de Maîtrise

Niveau Débutant Indicateurs :

- Comprend concepts de base : processus, fichier, permission, service
- Navigue filesystem (CLI : 'ls', 'dir', 'cd', 'find')
- Consulte logs basiques (Windows Event Viewer, Linux '/var/log/syslog')
- Identifie processus actifs ('ps', 'top', Task Manager)

Système Windows – Concepts de base :

- **Registre** : Base de données hiérarchique (HKEY_LOCAL_MACHINE, HKEY_CURRENT_USER, etc.)
- **Services** : Programmes arrière-plan ('services.msc')
- **Event Logs** : Journaux système (System, Security, Application)
- **UAC** : User Account Control (élévation privilèges)

Système Linux – Concepts de base :

- **Hiérarchie** : '/' root, '/home', '/var', '/etc', '/proc', '/sys'
- **Permissions** : 'rwxrwxrwx' (user, group, other)

- **Daemons** : Services arrière-plan (systemd, init.d)
- **Logs** : ‘/var/log/’ (syslog, auth.log, kern.log)

Exercice fondamental :

Identifier processus suspects sur système live :

```
# Linux
ps aux | grep -v "\[" | awk '{print $11}' | sort | uniq
# Analyser : processus avec noms inhabituels, chemins suspects

# Windows PowerShell
Get-Process | Select-Object Name, Path, Id |
  Where-Object {$_.Path -notlike "C:\Windows\*"} |
  Sort-Object Name
# Analyser : processus hors répertoires système
```

Niveau Intermédiaire Indicateurs :

- Maîtrise artefacts forensiques clés par OS
- Comprend mécanismes de persistance (autoruns, scheduled tasks, services)
- Analyse prefetch, ShimCache, Amcache (Windows)
- Interprète ‘/proc’, utmp, wtmp, bash_history (Linux)
- Extrait et analyse registre hors ligne (Registry Explorer)

Artefacts Windows essentiels :

Artefacts Linux essentiels :

Exercice intermédiaire :

Investiguer persistance sur Windows :

```
# PowerShell : Lister tous les points de persistance communs
Get-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"
Get-ItemProperty "HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"
Get-ScheduledTask | Where-Object {$_.State -eq "Ready"}
Get-Service | Where-Object {$_.StartType -eq "Automatic"}

# Analyser : chemins suspects, noms génériques (svchost, host, system)
# Vérifier signatures numériques des exécutables
Get-AuthenticodeSignature C:\suspect\malware.exe
```

Niveau Avancé Indicateurs :

- Expertise Windows Internals (Mark Russinovich) : comprend kernel, drivers, NTFS internals
- Maîtrise structures kernel Linux (task_struct, inode, dentry)

- Analyse mémoire kernel (dump kernel, WinDbg, crash)
- Développe parsers custom pour artefacts propriétaires
- Reconstitue activité complète système à partir artefacts multiples

Technique avancée : Analyse MFT (Master File Table) NTFS
Structure \$MFT :

- Fichier spécial contenant un record par fichier/dossier du volume
- Chaque record : 1024 bytes (généralement), numéro unique (MFT Entry)
- Attributs : \$STANDARD_INFORMATION (timestamps), \$FILE_NAME (nom, timestamps), \$DATA (contenu)

Timestamps MACB :

- **Modified** : Contenu fichier modifié
- **Accessed** : Fichier lu/exécuté
- **Created** : Fichier créé (ou copié vers volume)
- **Birth/Born** : MFT entry créée (NTFS)

Anomalies forensiques détectables :

1. **Timestamping** : Modification manuelle timestamps (outil comme TimeStomp)
 - **Détection** : \$STANDARD_INFORMATION modifiable, \$FILE_NAME (dans \$MFT) NON modifiable par APIs standard
 - Si SI_Modified ; FN_Modified → timestamping probable
2. **Fichiers supprimés** : MFT entry persiste après suppression (jusqu'à réallocation)
 - Parser \$MFT brut identifie entries supprimées (flag IN_USE=0)
 - Reconstruction nom fichier, timestamps, localisation cluster possible
3. **Timeline construction** : Tous timestamps de tous fichiers → chronologie complète activité filesystem

Outil pratique :

```
# Extraction MFT avec FTK Imager (GUI) ou Outils CLI
# Analyse avec MFTECmd (Eric Zimmerman)
MFTECmd.exe -f "C:\$MFT" --csv "C:\output" --csvf mft.csv
```

```
# Analyse CSV avec Python/Pandas
import pandas as pd
df = pd.read_csv('mft.csv')
```

```
# Détection timestomping
df['stomped'] = df['SI_Modified'] < df['FN_Modified']
stomped = df[df['stomped'] == True]
print(f"Fichiers timestompés : {len(stomped)}")

# Timeline
timeline = df[['FileName', 'SI_Modified', 'SI_Accessed',
               'SI_Created']].sort_values('SI_Modified')
```

Niveau Expert Indicateurs :

- Contributeur open-source outils forensiques (Volatility, Plaso, etc.)
- Recherche vulnérabilités OS pour comprendre vecteurs d'attaque
- Rédaction de plugins forensiques pour nouveaux artefacts
- Publication articles techniques (blogs, conférences SANS, DFRWS)
- Expertise reconnue (certifications multiples : GCFE, GCFA, EnCE)

4.1.5 Outils de Référence

Windows :

- **Sysinternals Suite** (Microsoft) : Process Explorer, Autoruns, TCPView, etc.
- **Eric Zimmerman Tools** : MFTECmd, PECmd, AppCompatCacheParser, etc.
- **Registry Explorer** : Analyse registre hors ligne
- **Event Log Explorer** : Analyse event logs (.evtx)

Linux :

- **Sleuth Kit** : Analyse filesystems (fls, icat, ils, etc.)
- **GRR Rapid Response** : Forensique live à distance
- **log2timeline/plaso** : Timeline generation multi-sources

Multi-plateforme :

- **Autopsy** : Interface graphique Sleuth Kit
- **X-Ways Forensics** : Commercial, très puissant
- **EnCase** : Standard industrie (commercial)

4.1.6 Erreurs Techniques Fréquentes

1. Analyser système live sans précautions :

- **Problème** : Modifie timestamps (Accessed), pollue artefacts
- **Solution** : Acquisition image disque d'abord, analyse hors ligne

2. Ignorer fuseaux horaires :

- **Problème** : Timestamps NTFS en UTC, Event Logs en local time
→ confusion timeline
- **Solution** : Normaliser tous timestamps en UTC, documenter time-zone système

3. Oublier artefacts secondaires :

- **Problème** : Focus Event Logs, ignore prefetch/ShimCache → activité manquée
- **Solution** : Checklist exhaustive artefacts par OS

4.1.7 Checklist de Validation CTf-1

Après analyse système :

Ai-je identifié l'OS exact (version, build, architecture) ?

Ai-je extrait et analysé tous artefacts de persistance (Run keys, services, tasks) ?

Ai-je parsé les artefacts d'exécution (prefetch, ShimCache, Amcache sur Windows) ?

Ai-je construit timeline multi-sources (MFT, Event Logs, bash_history) ?

Ai-je détecté anomalies (timestomping, fichiers cachés, processus suspects) ?

Ai-je documenté timezone et normalisé timestamps en UTC ?

Ai-je corrélié artefacts multiples pour corroborer hypothèses ?

4.1.8 Veille Technologique

Ressources essentielles :

- **Windows Internals (Russeinovich, Ionescu)** : Bible Windows, mise à jour régulière
- **13Cubed (YouTube)** : Tutoriels forensique Windows excellents
- **SANS Digital Forensics Blog** : Actualités artefacts, techniques

- [/r/computerforensics \(Reddit\)](#) : Communauté active

[Note : Les 17 autres compétences suivraient cette structure détaillée. Par contrainte de longueur, je présente ci-dessous la structure condensée des compétences restantes, puis conclus le chapitre.]

4.2 CTf-2 : Systèmes de Fichiers et Stockage

Définition : Maîtrise structures internes filesystems (NTFS, ext4, APFS, FAT, HFS+, exFAT), récupération fichiers supprimés, analyse slack space, carved data.

Concepts clés : Clusters, MFT/inode, journaling, timestamps, alternate data streams (ADS sur NTFS), file carving.

Outils : Sleuth Kit, Autopsy, R-Studio, PhotoRec, X-Ways.

4.3 CTf-3 : Analyse de la Mémoire Vive

Définition : Acquisition et analyse dumps mémoire RAM (processus, DLLs, handles, network connections, registry hives en mémoire, malware injection).

Concepts clés : Volatilité, VAD (Virtual Address Descriptor), PEB/TEB, handles, sockets.

Outils : Volatility Framework, Rekall, WinDbg, MemProcFS.

4.4 CTf-4 : Analyse du Trafic Réseau

Définition : Capture, filtrage et analyse paquets réseau (pcap), reconstruction sessions TCP, identification protocoles, détection activités malveillantes.

Concepts clés : TCP/IP, DNS, http/HTTPS, SMB, C2 traffic, exfiltration, covert channels.

Outils : Wireshark, tcpdump, NetworkMiner, Zeek (Bro), Suricata.

4.5 CTf-5 : Acquisition Forensique Conforme

Définition : Création images forensiques bit-à-bit, write-blocking, hash verification (MD5+SHA256), documentation chaîne de traçabilité.

Concepts clés : Write-blocker (hardware/software), formats images (E01, DD, AFF), integrity hashing.

Outils : FTK Imager, dd/dcfldd, Guymager, Tableau (hardware write-blockers).

4.6 CTf-6 : Timeline et Corrélation Multi-Sources

Définition : Construction chronologies exhaustives intégrant artefacts multiples (filesystem, logs, mémoire, réseau), corrélation événements, détection patterns.

Concepts clés : Super timeline, normalization timestamps, correlation, pivoting.

Outils : log2timeline/Plaso, Timesketch, Splunk, ELK Stack.

4.7 CTf-7 : Forensique Mobile

Définition : Acquisition et analyse iOS et Android (filesystem, bases SQLite, backup iTunes/iCloud, root/jailbreak forensics).

Concepts clés : Sandboxing, keychains (iOS), shared preferences (Android), SQLite (SMS, calls, apps data).

Outils : Cellebrite UFED, Oxygen Forensic, libimobiledevice, ADB (Android Debug Bridge).

4.8 CTf-8 : Forensique Cloud et Virtualisation

Définition : Investigation environnements cloud (AWS, Azure, GCP), containers (Docker, Kubernetes), machines virtuelles (VMware, Hyper-V).

Concepts clés : Snapshots VM, logs cloud (CloudTrail, Azure Monitor), forensique containers, persistance cloud.

Outils : AWS CloudTrail Analyzer, Azure Log Analytics, libvirt, vmware-mount.

4.9 CTf-9 : Reverse Engineering et Analyse de Binaires

Définition : Désassemblage et décompilation binaires (PE, ELF, Mach-O), compréhension assembly (x86, x64, ARM), reconstruction algorithmes.

Concepts clés : Assembly, calling conventions, imports/exports, obfuscation, packing.

Outils : IDA Pro, Ghidra, radare2, Binary Ninja, objdump, strings.

4.10 CTf-10 : Analyse Statique et Dynamique de Malware

Définition : Analyse comportementale malware en environnement contrôlé (sandbox), identification IOCs, techniques anti-analyse.

Concepts clés : Static analysis, dynamic analysis, sandbox, IOCs (Indicators of Compromise), YARA rules.

Outils : Cuckoo Sandbox, ANY.RUN, Joe Sandbox, PESTudio, YARA, VirusTotal.

4.11 CTf-11 : Détection et Analyse d’Intrusions

Définition : Identification compromissions via logs, SIEM, EDR, threat hunting, IOCs, TTP (Tactics, Techniques, Procedures).

Concepts clés : MITRE ATT&CK, IOCs, IOAs (Indicators of Attack), lateral movement, privilege escalation.

Outils : Splunk, ELK, Wazuh, OSSEC, Velociraptor, GRR.

4.12 CTf-12 : Cryptographie Appliquée et Cassage

Définition : Identification algorithmes crypto, cassage chiffrements faibles, password cracking, analyse implémentations cryptographiques.

Concepts clés : Symétrique (AES, DES), asymétrique (RSA, ECC), hashing (MD5, SHA), password cracking (brute-force, dictionary, rainbow tables).

Outils : Hashcat, John the Ripper, CyberChef, OpenSSL, cryptool.

4.13 CTf-13 : Analyse de Vulnérabilités

Définition : Identification vulnérabilités (CVE), classification CVSS, exploitation proof-of-concept, recommandations remediation.

Concepts clés : Buffer overflow, injection (SQL, XSS), CSRF, RCE, privilege escalation, CVE, CVSS.

Outils : Nessus, OpenVAS, Metasploit, Burp Suite, OWASP ZAP.

4.14 CTf-14 : Test de Pénétration et Red Teaming

Définition : Méthodologie offensive structurée (reconnaissance, exploitation, post-exploitation, reporting), simulation adversaire réaliste.

Concepts clés : Kill chain, PTES, OWASP Top 10, lateral movement, persistence, exfiltration.

Outils : Metasploit, Cobalt Strike, Empire, BloodHound, Mimikatz, nmap.

4.15 CTf-15 : Programmation Python pour Forensique

Définition : Développement scripts automatisant tâches forensiques (parsing logs, extraction artefacts, analysis), utilisation bibliothèques spécialisées.

Concepts clés : Parsing (struct, regex), manipulation fichiers, APIs forensiques (pytsk3, volatility), automation.

Outils/Libs : Python 3, pytsk3, volatility, dpkt (pcap), sqlite3, pandas, requests.

4.16 CTf-16 : Scripting PowerShell et Bash

Définition : Automatisation administrative et forensique Windows (PowerShell) et Unix/Linux (Bash), remote forensics.

Concepts clés : Cmdlets PowerShell, pipelines, remote execution (WinRM, SSH), parsing output, loops, conditionals.

Outils : PowerShell ISE, VS Code, bash, sed, awk, grep.

4.17 CTf-17 : Développement d'Outils Forensiques Sur Mesure

Définition : Création outils forensiques spécialisés (parsers propriétaires, extracteurs artefacts, analyseurs custom), architecture logicielle forensique.

Concepts clés : Architecture modulaire, formats binaires, performance, documentation, tests unitaires.

Langages : Python, C/C++, Rust, Go.

4.18 CTf-18 : Bases de Données et Requêtes

Définition : Extraction et analyse données structurées (SQL databases, SQLite, NoSQL), requêtage forensique, corrélation.

Concepts clés : SQL (SELECT, JOIN, WHERE), SQLite (smartphone apps, browsers), indexes, schemas.

Outils : sqlite3, DB Browser for SQLite, PostgreSQL, MySQL, MongoDB.

5 PARTIE III – Plan de Développement Technique et Ressources

5.1 Auto-Diagnostic Initial

Grille d'évaluation (mois 0) :

Pour chaque compétence CTf-1 à CTf-18, évaluer honnêtement :

- **0 :** Aucune connaissance
- **1 :** Débutant (concepts théoriques)
- **2 :** Intermédiaire (pratique guidée)
- **3 :** Avancé (autonomie complète)
- **4 :** Expert (contribution communauté)

Calcul score global : $Score = \sum_{i=1}^{18} Note_i$ (maximum 72)

Interprétation :

- 0-18 : Débutant (Phase 1 critique)
- 19-36 : Intermédiaire (Phase 2)
- 37-54 : Avancé (Phase 3 spécialisation)
- 55-72 : Expert (leadership, R&D)

5.2 Laboratoire Pratique Personnel

Configuration laboratoire forensique :

Hardware minimum :

- Laptop/Desktop : i7/Ryzen 7, 32 GB RAM, 1 TB SSD + 2 TB HDD
- Write-blocker USB (Tableau T8U ou similaire) : 200€

- Disques/clés USB tests divers capacités

Software essentiel (priorité open-source) :

- **OS hôte** : Windows 10/11 Pro ou Linux (Ubuntu/Kali)
- **Virtualisation** : VMware Workstation/VirtualBox
- **VMs** : Windows 10, Windows Server, Ubuntu, Kali Linux, REMnux (malware analysis)
- **Outils forensiques** : Autopsy, Volatility, Wireshark, FTK Imager, Eric Zimmerman Tools
- **Dev** : Python 3, VS Code, Git

Datasets pratique (légaux) :

- **CFReDS** (Computer Forensics Reference Data Sets) : Disques tests NIST
- **Digital Corpora** : Images forensiques académiques
- **SANS FOR508 challenges** : Exercices guided
- **Malware Traffic Analysis** : Pcaps malware réels

5.3 Certifications Professionnelles Recommandées

Trajectoire certification progressive :

Note sur coûts : Certifications GIAC coûteuses mais reconnues internationalement. Alternatives : certifications open-source (eLearnSecurity), formation académique (universités partenaires SANS).

5.4 Bibliographie Technique Essentielle

References

- [1] Russinovich, M., Solomon, D. A., & Ionescu, A. (2017). *Windows Internals, Part 1 & 2* (7th ed.). Microsoft Press.
- [2] Carrier, B. (2005). *File System Forensic Analysis*. Addison-Wesley.
- [3] Case, A., & Richard, G. G. (2017). *Memory Forensics : The Path Forward*. Digital Investigation, 20, 23-33.
- [4] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379-423.
- [5] Von Neumann, J. (1945). *First Draft of a Report on the EDVAC*. University of Pennsylvania.

- [6] Schneier, B. (2015). *Applied Cryptography* (2nd ed.). Wiley.
- [7] Erickson, J. (2008). *Hacking : The Art of Exploitation* (2nd ed.). No Starch Press.
- [8] Sikorski, M., & Honig, A. (2012). *Practical Malware Analysis*. No Starch Press.
- [9] Sanders, C. (2018). *Practical Packet Analysis* (3rd ed.). No Starch Press.
- [10] Carvey, H. (2018). *Windows Registry Forensics* (2nd ed.). Syngress.

5.5 Ressources en Ligne et Communautés

Formations gratuites/abordables :

- **13Cubed (YouTube)** : Tutoriels Windows forensics excellents
- **DFIR Training** : Cours forensique gratuits/payants
- **Cybrary** : Plateforme e-learning sécurité
- **TryHackMe, HackTheBox** : Labs pratiques

Blogs techniques :

- SANS Digital Forensics & Incident Response
- Volatility Labs Blog
- Belkasoft Blog
- Magnet Forensics Blog

Communautés :

- /r/computerforensics, /r/netsec (Reddit)
- SANS DFIR Discord/Slack
- Forensic Focus Forums

5.6 Message Final : La Maîtrise Technique comme Voyage Perpétuel

L'excellence technique en investigation numérique n'est jamais acquise définitivement. C'est un voyage d'apprentissage continu face à un paysage technologique en évolution rapide.

Trois vérités fondamentales :

1. **Profondeur & Largeur (initialement)** : Mieux maîtriser 6 compétences solidement que 18 superficiellement. Phase 1 : socle solide.

2. **Pratique délibérée** : 10,000 heures ne suffisent pas. Nécessite pratique *délibérée* : sortir zone de confort, recevoir feedback, ajuster.
3. **Communauté = multiplicateur** : S'engager dans communauté (forums, conférences, open-source) accélère apprentissage 3-5×. Partager connaissances = consolider maîtrise.

Citation de Bruce Schneier (cryptographe) :

Complexity is the worst enemy of security.

Applicable à l'apprentissage : Commencez simple, construisez progressivement, maîtrisez fondamentaux avant sophistication.

Roadmap personnalisée :

Utilisez les grilles d'auto-évaluation, identifiez vos lacunes prioritaires, construisez plan 36 mois réaliste. Réévaluez tous les 6 mois, ajustez trajectoire.

L'investigation numérique d'excellence conjugue rigueur scientifique (Partie I), maîtrise technique (Partie II) et compétences humaines (chapitre précédent). Vous avez maintenant la carte. À vous de parcourir le territoire.

The only way to do great work is to love what you do. — Steve Jobs

Bonne investigation.

Table 1: Cartographie fondements scientifiques → compétences techniques

Fondement	Concept clé	Applications aux compétences techniques (CTf)
Architecture von Neumann	Programme = données en mémoire	CTf-1 : Comprendre où artefacts résident (registres, RAM, disque) CTf-3 : Analyse mémoire (dumps, volatility) CTf-8 : Reverse malware (code stocké comme données)
Hiérarchie mémoire	Cache, RAM, disque (vitesse vs persistance)	CTf-2 : Priorisation acquisition (volatil d'abord) CTf-3 : Artefacts cache CPU/disque révèlent activité récente
Systèmes d'exploitation	Abstractions (processus, VM, filesystem)	CTf-1 : Maîtriser Windows/Linux/macOS internals CTf-2 : Comprendre NTFS, ext4, APFS pour forensique disque CTf-3 : Reconstruction mapping mémoire virtuelle
Théorie information (Shannon)	Entropie mesure aléatoire/information	CTf-6 : Détection chiffrement (entropie max) CTf-8 : Détection malware packé (sections haute entropie)
Complexité (P/NP)	Certains problèmes intrinsèquement difficiles	CTf-6 : Sécurité crypto repose sur NP-hardness CTf-8 : Obfuscation peut être NP-complet (désobfuscation difficile)
Arithmétique modulaire	Base RSA, Diffie-Hellman	CTf-6 : Comprendre cryptosystèmes modernes Analyse vulnérabilités implémentations crypto
Courbes elliptiques	ECC (Bitcoin, TLS 1.3, Signal)	CTf-6 : Forensique blockchain, analyse protocoles modernes Comprendre signatures ECDSA
Fonctions de hachage	Hash unique = empreinte, détection collision	CTf-2 : Intégrité preuves (SHA-256) CTf-6 : Identification fichiers (MD5 obsolète, utiliser SHA-256+) Détection altération (effet avalanche)
Principe de Locard	Toute action laisse traces	TOUTES CTf : Fondement forensique. Aucune action sans artefacts. Multi-sources permettent corroboration. Anti-forensique limité par impossibilité perfection.
Chaîne de traçabilité	Documentation continue preuve	CTf-2 : Acquisition conforme (hash, formulaires, scellés) CTf-15 : Méthodologie garantissant recevabilité juridique
Méthode hypothético-déductive	Hypothèse → Prédiction → Test → Évaluation	CTf-15 : Investigation scientifique structurée Éviter biais confirmation (falsification poppérienne) Documentation reproductible
Daubert standard	5 critères fiabilité scientifique	CTf-14, CTf-15 : Utiliser méthodes testables, peer-reviewed Documenter taux erreur, méthodologie Défendable en contre-interrogatoire
Modèle OSI/TCP-IP	Encapsulation couches, protocoles	CTf-4 : Analyse multi-couches pcap Reconstruction sessions TCP Comprendre protocoles applicatifs (http, DNS, SMB)
TCP	Connexion fiable, handshake, séquences	CTf-4 : Reconstruction conversations (Follow TCP Stream) Détection port scanning (SYN floods) Covert channels
DNS	Résolution noms, hiérarchie, types records	CTf-4 : Identification C2 malware via DNS Détection DNS tunneling (exfiltration TXT records) DGA detection (entropie domaines)

Table 2: Référentiel des 18 compétences techniques essentielles

Bloc	Code	Compétence	Ancrages théoriques
4* Fondamental	CTf-1	Architecture et internals des systèmes (Windows, Linux, macOS)	von Neumann, OS theory
	CTf-2	Systèmes de fichiers et stockage (NTFS, ext4, APFS, HFS+)	Hiérarchie mémoire, persistance
	CTf-3	Analyse de la mémoire vive (RAM forensics)	Volatilité, mémoire virtuelle
	CTf-4	Analyse du trafic réseau et protocoles	TCP/IP, OSI, DNS
4* Forensique	CTf-5	Acquisition forensique conforme (imaging, write-blocking)	Locard, chaîne traçabilité
	CTf-6	Timeline et corrélation multi-sources	Méthodologie scientifique
	CTf-7	Forensique mobile (iOS, Android)	Architecture mobile, sandboxing
	CTf-8	Forensique cloud et virtualisation	Architecture distribuée
3* Analytique	CTf-9	Reverse engineering et analyse de binaires	Assembly, architecture CPU
	CTf-10	Analyse statique et dynamique de malware	Obfuscation, comportement
	CTf-11	Détection et analyse d'intrusions	Patterns, anomalies
4* Sécurité	CTf-12	Cryptographie appliquée et cassage	Théorie des nombres, ECC
	CTf-13	Analyse de vulnérabilités	Types vulnérabilités, exploitation
	CTf-14	Test de pénétration et red teaming	Méthodologie offensive
4* Automatisation	CTf-15	Programmation Python pour forensique	Algorithmique, structures données
	CTf-16	Scripting PowerShell et Bash	Automatisation système
	CTf-17	Développement d'outils forensiques sur mesure	Architecture logicielle
	CTf-18	Bases de données et requêtes (SQL, logs)	Modèle relationnel, indexation

Table 3: Artefacts forensiques Windows critiques

Artefact	Localisation	Information forensique
Prefetch	C:\Windows\Prefetch*.pf	Historique exécution programmes (dernier run, nb exécutions, fichiers/DLLs chargées)
ShimCache	Registre : HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\AppCompatCache	Programmes exécutés (pas nécessairement les chemins complets, timestamps)
Amcache	C:\Windows\appcompat\Programs\Amcache.hint	Installation programmes, SHA1 hashes, meta-data exécutables
SRUM	C:\Windows\System32\sru\SRUDB.dat	Utilisation réseau par app, durée exécution, bytes envoyés/reçus
\$MFT	Racine volume NTFS	Master File Table : tous fichiers/dossiers, timestamps, attributs
Event Logs	C:\Windows\System32\winevt\Logs*.evtx	Événements système : 4688 (création processus), 4624/4625 (logon), 7045 (nouveau service), etc.

Table 4: Artefacts forensiques Linux critiques

Artefact	Localisation	Information forensique
bash_history	<code>/.bash_history</code>	Commandes shell exécutées par utilisateur
auth.log	<code>/var/log/auth.log</code> (Debian) ou <code>/var/log/secure</code> (RHEL)	Authentifications (SSH, sudo), succès/échecs
wtmp/utmp	<code>/var/log/wtmp</code> , <code>/var/run/utmp</code>	Sessions utilisateurs (login/logout, IPs sources)
lastlog	<code>/var/log/lastlog</code>	Dernier login par utilisateur
syslog	<code>/var/log/syslog</code>	Messages système généraux
/proc	Pseudo-filesystem	Processus actifs (<code>/proc/[PID]/</code>) : <code>cmdline</code> , <code>environ</code> , <code>exe</code> , <code>maps</code> , <code>fd</code>
cron logs	<code>/var/log/cron</code>	Exécutions tâches planifiées

Table 5: Roadmap certifications forensiques

Phase	Certification	Coût	Difficulté
Phase 1 (12 mois)	GIAC GCFE (Certified Forensic Examiner) CompTIA CySA+ (optionnel)	7k€ 400€	Intermédiaire Débutant
Phase 2 (24 mois)	GIAC GCFA (Certified Forensic Analyst) EnCE (EnCase Certified Examiner)	7k€ 3k€	Avancé Intermédiaire
Phase 3A (Malware)	GIAC GREM (Reverse Engineering Malware)	7k€	Expert
	GIAC GNFA (Network Forensic Analyst)	7k€	Avancé
Phase 3B (Pentest)	OSCP (Offensive Security Certified Prof.) GIAC GPEN (Penetration Tester)	1k€ 7k€	Avancé Avancé
Phase 3C (Forensique)	GIAC GCFA + spécialisations Certifications vendors (Cellebrite, Magnet)	Variable Variable	Expert Variable